23k-0066

Q2

generation (1) ⟶ initial population

[[2, 2, 1, 2, 1, 2, 1], [2, 1, 1, 3, 1, 3, 2],
[1, 2, 1, 1, 3, 1, 3], [2, 2, 2, 2, 1, 1, 3],
[1, 2, 2, 3, 3, 2, 3], [2, 2, 3, 2, 1, 2, 2]
]

Fitness calculation

[2, 2, 1, 2, 1, 2, 1]

Task1 → Facility2 → 5 x 12 = 60
T2 → F2 → 8 x 14 = 112
T3 → F1 → 4 x 8 = 32
T4 → F2 → 7 x 10 = 70
T5 → F1 → 6 x 14 = 84
T6 → F2 → 3 x 8 = 24
T7 → F1 → 9 x 11 = 99

Time used:

F1 : ~~3+5+7 = 15~~ 4+6+9 = 19
F2 : ~~2+4+6+11 = 13~~ 5+8+7+3 = 23
F3 : 0

All within capacity so no penalty

Total cost: 60 + 112 + 32 + 70 + 84 + 24 + 99 = ~~459~~ 481

Fitness : ~~1/459 = 0.00217~~ 1/481 = 0.00207

Same for rest 6

Individual	cost	Penalty	Fitness
1	481	0	0.00207
2	525	0	0.00190
3	494	0	0.00202
4	506	0	0.00197
5	502	0	0.00198
6	486 (without penalty) 2486 (with penalty)	2000	0.00040

Roulette wheel

total Fitness = 0.01035

Individual 1 → 0.00207/0.01035 = 0.2002 = 20.02

I2 → 18.35

I3 → 19.50

I4 → 19.04

I5 → 19.19

I6 → 3.88

Cross over:

Selecting a pair for cross over using roulette ~~whete~~ wheel.

Individual 4 → [2, 2, 2, 2, 1, 1, 3]
Individual 2 → [2, 1, 1, 3, 1, 3, 2]

crossover at point 2

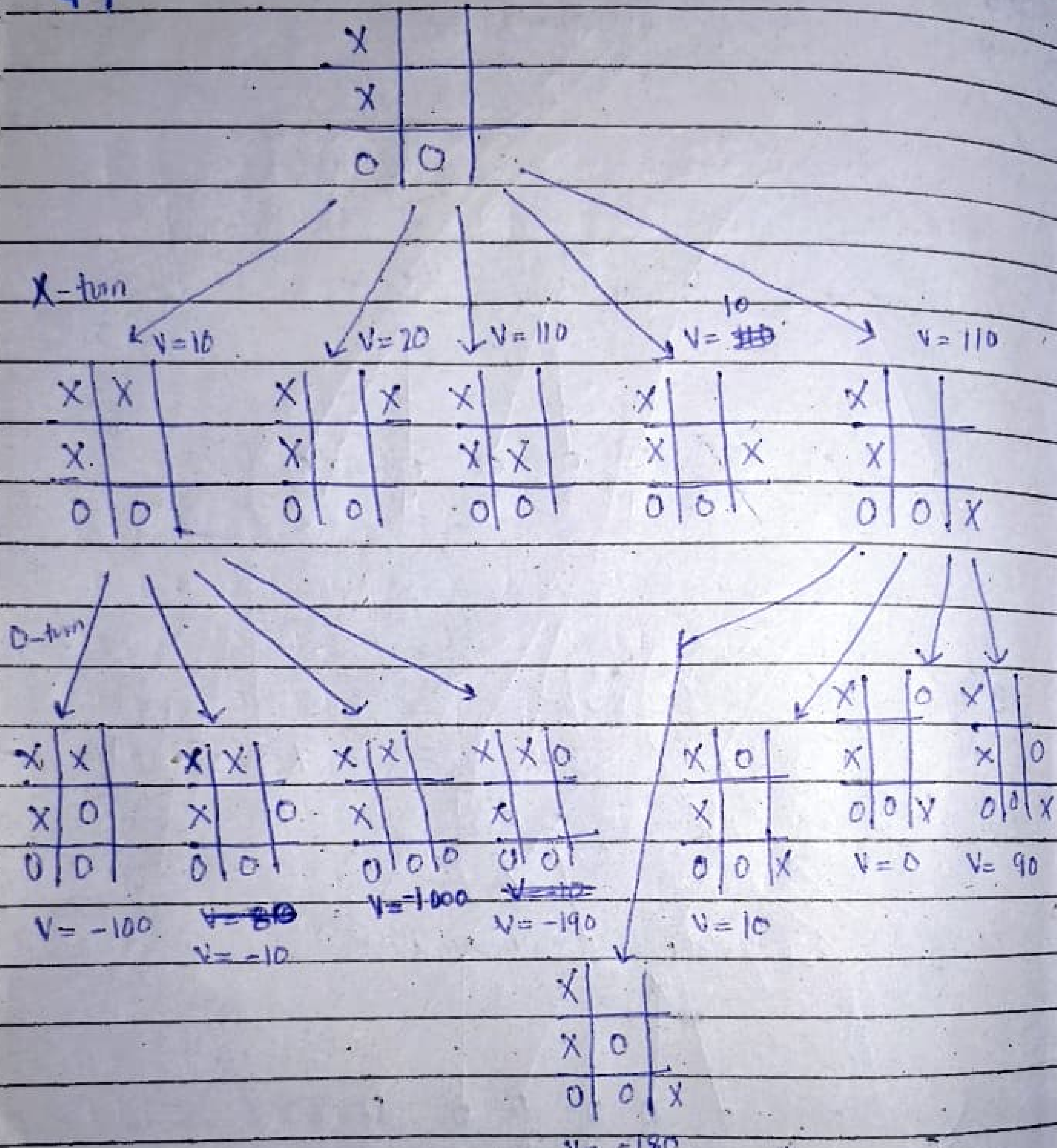
c1 : [2, 2, 1, 3, 1, 3, 2]
c2 : [2, 1, 2, 2, 1, 1, 3]

Mutation:

swapping positions 6 and 0 in c2
mutated to : [3, 1, 2, 2, 1, 1, 2]
swapping positions 2 and 5 in c1
mutated to : [2, 2, 3, 3, 1, 1, 2]

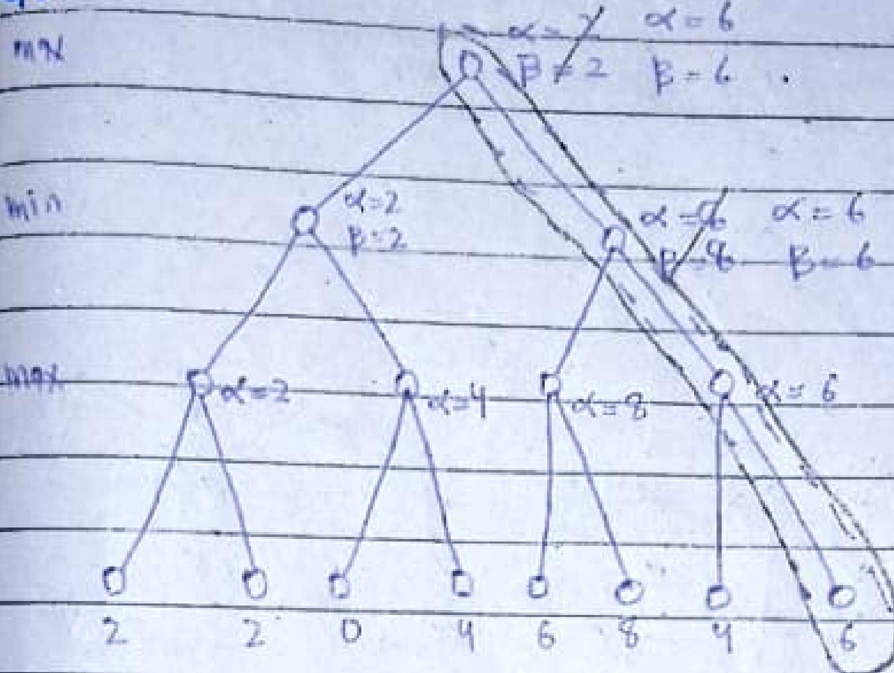
In this way we will keep generating all of this for the next generation.

Q4

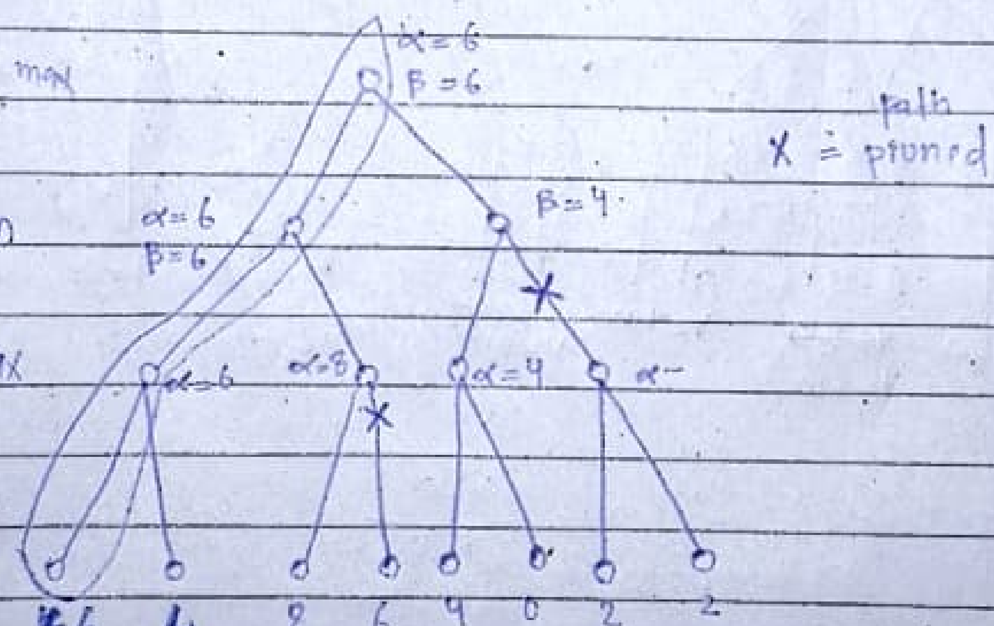


X will choose the right most block to maximize

Q5


max
min
max
α = 6
β = 6
α = 2
β = 2
α = 6
β = 6
α = 2
α = 4
α = 8
α = 6
2
2
0
4
6
8
4
6

B


max
min
max
α = 6
β = 6
α = 6
β = 6
β = 4
α = 6
α = 8
α = 4
6
4
8
6
4
0
2
2
path
X = pruned

Q6

(a)

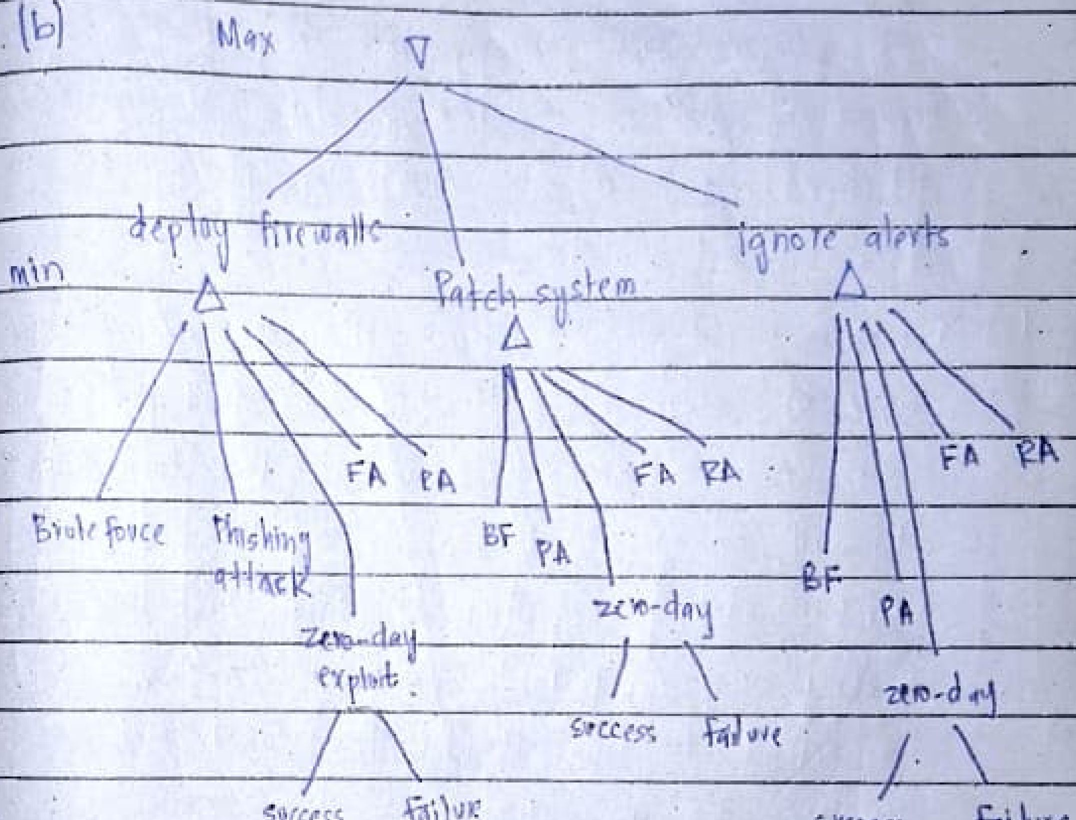
(1) Max (defender): The AI-based IDS whose goal is to protect the network from cyber attacks
Min (attacker): Hacker trying to breach the system using various methods
Both players aim for opposite objectives i.e defender's goal is to minize damage while attacker's goal is to maximize it

(2) Defender uses logical decision-making by evaluating different strategies. It considers past patterns and known threats to decide whether to patch, Firewall, or ignore alerts.
Attacker adopts such attack strategies that are either stealthy or unpredictable. It may also deceive the defender to make it (defender) wastes its resources by fake attacks.

(3) Probabilistic attacks (like zero-day exploits) introduce randomness into the game, making it complex. Unlike deterministic attacks where outcomes are predictable, probabilistic attacks have uncertain results. Because of this uncertainity, defender can't always assume the worst like in Minimax. Instead

defender can use Expectimax, which calculates the average expected outcome based on probabilities.
For Example: if a zero-day exploit has a 50% chance of success, defender will weigh both success & failure chances when deciding how to respond.

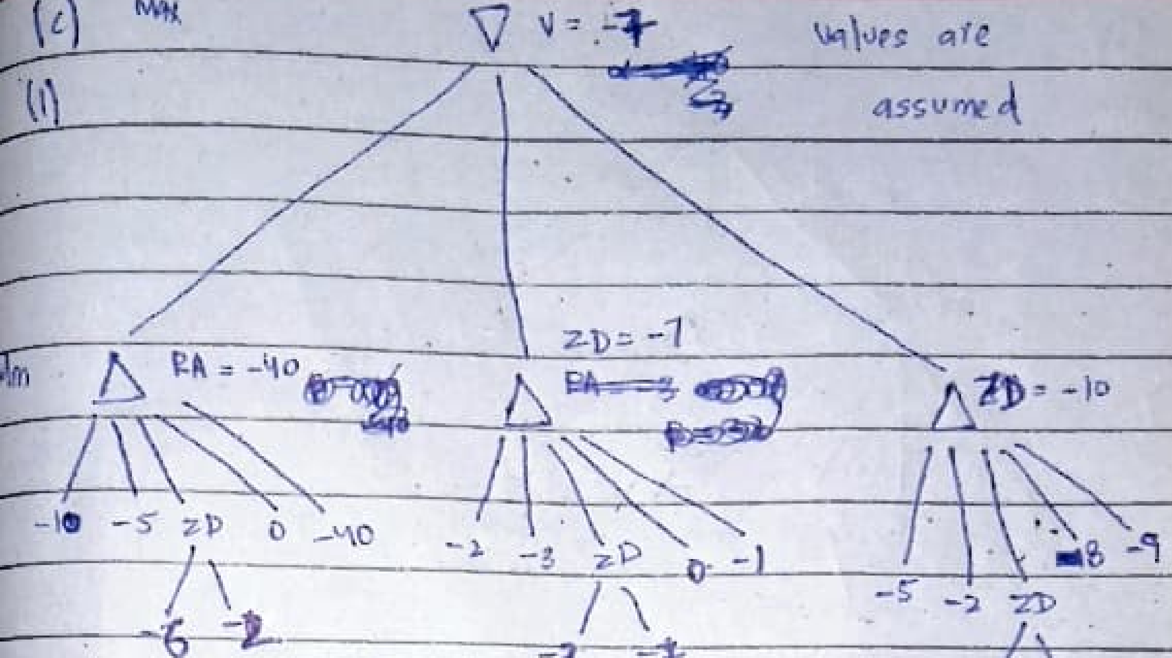
(b)



(c)

(1)

values are assumed



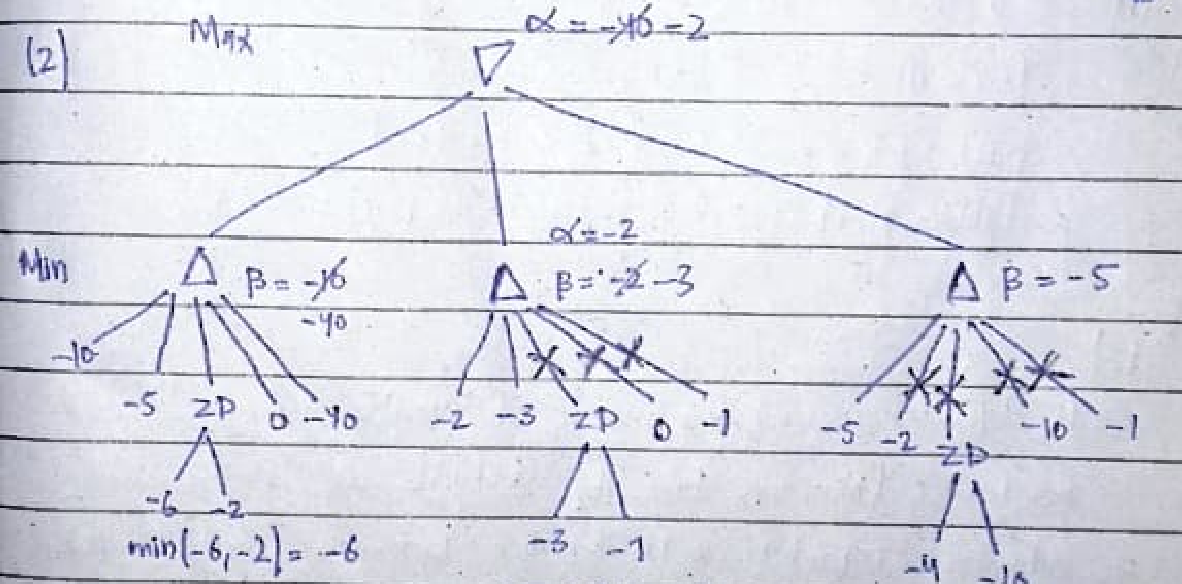
min(-6, -2) = -6

min(-3, -7) = -7

min(-4, -10) = -10

Defender → best move → patch system

(2)



min(-6, -2) = -6

min(-3, -7) = -7

min(-4, -10) = -10

Defender's best move is to patch system

(d)

(1) 50% success rate

failure / success

Firewall → $0.5(-2) + 0.5(-6) = -4$

patch system → $0.5(-7) + 0.5(-3) = -5$

Ignore alert → $0.5(-10) + 0.5(-4) = -7$

(2)

If defender uses Expectimax, it won't always assume worst case (Minimax does it) but it will average out outcomes. Defender would balance between cost of defence and the chances of attack success. For example, defender may choose not to fully defend against a low-probability attack if the expected value is not very bad.